MyTix User Manual

Console ticketing platform - operational guide

Purpose

MyTix is a database-backed console application for browsing events, buying and managing tickets, resale, reviews, organizer administration, and analytical reports. This manual describes the workflows currently exposed by the application.

Before you begin

• Configure the database connection values in **.env** (DB_URL, DB_USER, and DB_PASSWORD).

• Run **./run.sh** in WSL/Linux, from the project root. The script compiles the Java sources against the MySQL connector, OpenNLP, and SLF4J jars in the project root and starts MyTix.

• Create the database with **sql/schema.sql** and load the sample data with **sql/load.sql** before starting the application.

• Date format is YYYY-MM-DD; date/time format is YYYY-MM-DDTHH:MM; month format is YYYY-MM; prices use decimals such as 49.99.

Start menu

Choice	What it does
1	Browse as a guest: search events and run reports.
2 / 3	Log in as a customer or organizer using the account ID or email.
4 / 5	Create a customer or organizer account.
0	Exit the application.

Account setup and sign-in

Customer registration asks for name, email, address, date of birth, cardholder name, card number, and expiry. Organizer registration asks for the first four identity fields. Users must be at least 18. After successful registration, the new account is opened automatically. Existing users can sign in by their account ID or matching email and account type.

Guest browsing

Option	Use
Q1	Find performances near a latitude/longitude within a radius; choose a distance or price ranking and direction.
Q2 / Q3	Search by postal code or exact venue address.

Option	Use
Q4	Search within a date/time range with a minimum number of available tickets.
Q4b / Q4c / Q4d	Same date/time-and-availability refinement as Q4, chained onto a Q1 location search, a Q2 postal-code search, or a Q3 exact-address search respectively.
Q5	Combine city, segment, genre, time, price, availability, and seating-type filters. Blank inputs mean no filter.
Q6	View a performance seat-map summary.
Q7	Request the best available seats for a performance, quantity, and optional budget.

Guest reports

Guest access also provides city/venue revenue, taxonomy and location counts, organizer revenue rankings, potential scalpers, customer-order rankings, cancellation leaders, resale activity, and frequently occurring comment phrases. Reports requiring a date range prompt for start and end dates. Sell-through has three variants: by price tier, rolled up by performance, and (for a given month) which performances sold out or sold under a quarter of capacity, by city.

Customer workflows

Choice	Workflow
1	Reserved booking: choose a performance, payment method, section, row, and one or more seat numbers.
2	General-admission booking: choose a performance, section, payment method, and quantity.
3	Cancel one or more owned tickets by comma-separated ticket numbers.
4	List an owned ticket for resale with an asking price.
5	Withdraw an active resale listing by listing number.
6	Buy an active resale listing using the current customer payment method.
7	Review an attended event: enter its exact title, comment, event rating, and venue rating (both 1-5).

Customer tips

• Where a performance, venue, section, or tier allows an optional ID, press Enter to identify it by its exact name/address and date/time instead.

• For seat or ticket lists, separate numbers with commas, for example: 12, 13, 14.

• Use the exact event title when submitting a review. The system accepts one review only after attendance has been established.

Organizer workflows

Choice	Workflow
1	Create an event: enter segment, genre, title, description, and resale-cap multiplier (for example, 1.20).
2	Add a future performance to an owned event using a venue, performance name, and date/time.
3	Define a named price tier for an owned performance.
4	Assign a section to a price tier for an owned performance.
5	Change an owned event’s resale-cap multiplier.
6	Change a price tier, only while its performance is still scheduled and in the future, and only when no ticket in that tier has been sold.
7	Block an unsold reserved seat, optionally with a reason (e.g. obstructed view, production equipment).
8	Cancel an owned scheduled performance.

Important organizer rules

• A performance must be scheduled in the future and belongs to an event owned by the logged-in organizer.

• A blocked seat cannot already be sold.

• Cancelling a scheduled performance records refunds for active tickets, withdraws active resale listings, and marks the tickets as cancelled by organizer.

Troubleshooting

• “No matching record found” usually means the supplied ID, exact name, address, or time does not match a database record.

• “Price cannot change after a sale” protects price consistency after sales in that tier.

• If startup fails, verify the .env database settings, that MySQL is running, and that the three jars named in run.sh (MySQL connector, OpenNLP, SLF4J) are present in the project root.

• R9 additionally requires en-token.bin, en-pos-maxent.bin, and en-chunker.bin in the models/ directory.

• Use 0 in any menu to return or log out.

MyTix System Limitations and Improvement Opportunities

Current-state assessment and practical roadmap

Scope

This document assesses the implemented Java/MySQL console application. It separates present capabilities from gaps visible in the user interface and source implementation, so improvements can be planned without overstating current functionality.

Current strengths

• Clear role separation for guests, customers, and organizers.

• Relational constraints and transactions support account creation, booking, resale, and performance cancellation.

• The system supports reserved and general-admission sales, resale price caps, reviews, blocking, refunds, searches, and reports.

• Organizer ownership checks are applied to event and performance management operations.

Key limitations

Area	Current limitation	Impact
Authentication	Login matches an ID or email and account type; no password, session control, or recovery flow is present.	Anyone knowing an identifier may be able to access an account.
Payments	Card data is collected and stored directly; no payment gateway, tokenization, encryption evidence, or payment status workflow is exposed.	High security and compliance risk; payments are not production-ready.
Interface	The application is command-line only and relies on exact input formats, IDs, names, and dates.	Harder for nontechnical users; invalid input can interrupt flows.
Discovery	Several lookups require exact titles, names, or addresses; results are printed rather than navigable.	Users may struggle to find the intended record.
Operations	No visible notifications, ticket delivery/QR codes, check-in, waitlist, or customer support workflow.	Incomplete end-to-end event attendance experience.
Data quality	Some management tasks are single-item operations, such as assigning one section/tier at a time.	Setup can be slow and mistakes are harder to detect.
Reporting	Reports print raw rows to the console without export, charts, scheduled delivery, or access controls.	Limited usefulness for business decision-making.
Recommendations	Pricing suggestions use simple historical averages and limited comparable criteria.	Recommendations can be inaccurate or biased when demand differs.

Recommended improvements

Priority	Improvement	Expected benefit
1 - Security	Implement password hashing, account verification, role-based authorization, secure sessions, audit logging, and least-privilege database credentials. Replace stored card details with a PCI-compliant payment provider and tokens.	Protects users and makes the platform suitable for real accounts and payments.
2 - Reliability	Add centralized input validation, user-friendly error handling, idempotent booking/resale operations, transaction isolation/concurrency controls, and automated backup/recovery procedures.	Reduces crashes, duplicate sales, and operational loss.
3 - Usability	Build a web or mobile interface with searchable listings, form validation, seat maps, account pages, and clear confirmations. Support partial matching and filters that preserve user choices.	Makes core workflows accessible and discoverable.
4 - Ticket lifecycle	Issue digital tickets with QR/barcodes, add venue check-in/scanning, email confirmations, alerts for cancellations/refunds/resale activity, and configurable cancellation policies.	Completes the customer journey and improves event operations.
5 - Organizer tools	Provide multi-section configuration, templates, preview/validation before publishing, inventory dashboards, and exports.	Speeds event creation and lowers setup errors.
6 - Analytics	Add dashboards, CSV/PDF export, report scheduling, data retention rules, and explainable demand/pricing models based on broader comparables.	Turns stored data into actionable insights.

Suggested delivery sequence

• First: secure authentication and payment handling; add automated tests for booking, cancellations, and resale.

• Next: improve concurrency protection and exception handling, then provide a user-facing web interface.

• Then: add ticket delivery/check-in and notifications.

• Finally: enhance reporting, forecasting, and organizer optimization after trustworthy operational data has accumulated.

Success measures

Track successful booking completion, failed/duplicate transaction rate, support requests per booking, time to configure an event, refund completion time, check-in rate, and report adoption. Security work should also include periodic access reviews, dependency updates, and tested recovery exercises.

Conclusion

MyTix has a solid academic prototype foundation: its relational workflows cover the principal ticketing concepts. The highest-value next steps are safeguarding identities and payments, protecting transactions under load, and replacing the console-only experience with a guided interface.